

Dark UI | Dart | Flutter

Daily Water & Hydration Tracker App

تكليف عملي في تطوير تطبيقات Flutter

تصميم الواجهات • التفاعل مع المستخدم • التنقل بين الشاشات

المستوى: **مبتدئ إلى متوسط** الدرجة: **100** التسليم: **GitHub**

فهرس المحتويات

1	نظرة عامة على المشروع
2	أهداف التعلم
3	متطلبات المشروع (الشاشة الأولى والثانية)
4	المنطق البرمجي والحسابات
5	التصميم ونظام الألوان
6	مخططات Wireframe للشاشتين
7	الـ Widgets المطلوب تطبيقها
8	الهيكلية البرمجية المقترحة
9	التحديات الإضافية Bonus Challenges
10	معايير التقييم Rubric
11	متطلبات التسليم وقائمة التحقق
12	إرشادات التنفيذ والأخطاء الشائعة

كيف تستخدم هذا المستند؟

اقرأ المستند كاملًا قبل البدء بالبرمجة، وابدأ بمخططات Wireframe في القسم 6 لتفهم توزيع العناصر. المستند لا يتضمن الحل الكامل، بل مقتطفات قصيرة توضّح الفكرة فقط؛ الهدف أن تنفّذ المشروع بنفسك.

تنبيه مهم

لا يُشترط في هذا التكليف استخدام قواعد البيانات أو Firebase أو REST APIs أو مكتبات إدارة الحالة الخارجية. المطلوب فقط ما درسته في المحاضرات: Widgets الأساسية، و setState()، والتنقل بين الشاشات.

1 نظرة عامة على المشروع

اسم المشروع: Daily Water & Hydration Tracker App

المطلوب تطوير تطبيق Flutter يساعد المستخدم على إدخال بعض البيانات الأساسية (الجنس، ساعات النشاط، الوزن، العمر)، ثم يحسب **تقديرًا** لاحتياجه اليومي من الماء، ويعرض النتيجة مع نص توجيهي مبسّط في شاشة منفصلة.

يجب أن تكون الواجهة حديثة وبسيطة وجذابة، وتعتمد على **Dark Theme** مع اللون السماوي الذي يرمز إلى الماء.

ما هو المطلوب وما هو غير المطلوب؟

- المطلوب: مشروع تعليمي يركّز على تصميم الواجهات، والتعامل مع البيانات، والتفاعل مع المستخدم، والتنقل بين الشاشات.
- غير المطلوب: بناء تطبيق طبي متكامل. الناتج رقم تقديري لأغراض التدريب وليس توصية صحية شخصية.

البند	التفاصيل
مستوى التكليف	مبتدئ إلى متوسط
التقنيات	Flutter و Dart فقط (بدون مكتبات خارجية إلزامية)
عدد الشاشات	شاشتان: Input Screen و Result Screen
الدرجة الكلية	100 درجة (+ تحديات اختيارية لا تؤثر سلبًا على التقييم)
طريقة التسليم	رابط GitHub Repository + README.md + صورتان للشاشتين

2 أهداف التعلم

يهدف المشروع إلى تطبيق المفاهيم التالية عمليًا:

1. إنشاء مشروع Flutter وتنظيم الملفات.
2. فهم الفرق بين StatelessWidget و StatefulWidget.
3. استخدام setState() لتحديث واجهة المستخدم.
4. التعامل مع المتغيرات والأنواع الأساسية في Dart.
5. إنشاء واجهات باستخدام Scaffold و AppBar و Container و Column و Row و Text.
6. تنظيم العناصر باستخدام Expanded و Flexible و Padding و SizedBox.
7. استخدام GestureDetector أو InkWell للتفاعل مع البطاقات.
8. استخدام Slider لتغيير قيمة النشاط البدني.
9. استخدام الأزرار لتغيير الوزن والعمر.
10. استخدام Navigator.push() و Navigator.pop() للتنقل بين الشاشات.
11. تمرير البيانات من شاشة الإدخال إلى شاشة النتيجة.
12. استخدام الدوال لإجراء العمليات الحسابية.
13. تطبيق الألوان والخطوط والحواف والتصميم المتجاوب.
14. تقسيم الواجهة إلى Widgets صغيرة قابلة لإعادة الاستخدام.

3 متطلبات المشروع

الشاشة الأولى: Input Screen

يجب أن تحتوي الشاشة على العناصر التالية (الأرقام تطابق مخطط Wireframe في القسم 6):

#	القسم	المتطلبات
1	رأس الصفحة	• عنوان التطبيق. • أيقونة مناسبة للماء. • تصميم داكن منظم.
2	اختيار الجنس	• بطاقتان: Male و Female، لكل منهما أيقونة ونص. • يتغير لون البطاقة عند تحديدها. • لا يمكن تحديد الخيارين في الوقت نفسه.
3	مستوى النشاط البدني	• عنوان واضح. • Slider لاختيار عدد ساعات النشاط اليومي، بنطاق مناسب مثل 0 إلى 8 ساعات. • عرض القيمة الحالية للمستخدم.
4	الوزن	• بطاقة بعنوان Weight تعرض الوزن الحالي بالكيلوجرام. • زر دائري (+) للزيادة وزر دائري (-) للإنقاص. • منع القيم غير المنطقية أو السالبة.
5	العمر	• بطاقة بعنوان Age تعرض العمر الحالي بالسنوات. • زرا (+) و (-) للتعديل. • حد أدنى وأقصى مناسبان للقيم التعليمية.
6	زر الحساب	• زر عريض أسفل الشاشة بعنوان Calculate Water Intake. • عند الضغط عليه: التحقق من المدخلات ثم الحساب. • الانتقال إلى شاشة النتيجة مع تمرير البيانات اللازمة.

الشاشة الثانية: Result Screen

#	العنصر	المتطلبات
1	العنوان	عنوان مثل Your Daily Water Intake.
2	بطاقة النتيجة	بطاقة مركزية بارزة تعرض النتيجة، مثل: 2.95 L / Day.
3	توضيح التقدير	توضيح أن القيمة **تقديرية** وليست توصية طبية شخصية.
4	نص توجيهي	نص مبسّط يعتمد على النتيجة، دون ادعاءات طبية غير موثقة (انظر القسم 4).
5	ملخص المدخلات	عرض مختصر للوزن والعمر وساعات النشاط.
6	زر Re-calculate	للعودة إلى الشاشة الأولى باستخدام Navigator.pop().

حافظ على تناسق الألوان والتصميم بين الشاشتين.

4 المنطق البرمجي والحسابات

معادلة تعليمية وليست معيارًا طبيًا

المعادلة التالية **مبسّطة لأغراض التدريب فقط** وليست معيارًا طبيًا معتمدًا. الاحتياجات الفعلية للماء تختلف من شخص لآخر حسب عوامل عديدة، لذلك لا تُعرض النتيجة على أنها كمية مناسبة للجميع.

خطوات الحساب

```
baseMl = weightKg × 35 // base need
activityMl = activityHours × 250 // activity extra
totalMl = baseMl + activityMl // total in ml
litres = totalMl / 1000 // convert to litres
```

مثال محلول

الوزن 70 كجم، وعدد ساعات النشاط 2 ساعة:

```
baseMl = 70 × 35 = 2450 ml
activityMl = 2 × 250 = 500 ml
totalMl = 2450 + 500 = 2950 ml
litres = 2950 / 1000 = 2.95 L / Day
```

حالات اختبار للتأكد من صحة الدالة

الوزن (kg)	النشاط (h)	الحساب بالمليلتر	النتيجة (L / Day)
50	0	1750 + 0 = 1750	1.75
60	8	2100 + 2000 = 4100	4.10
70	2	2450 + 500 = 2950	2.95
90	4	3150 + 1000 = 4150	4.15

قواعد مهمة في الحساب

- نفّذ الحساب داخل **دالة مستقلة** وأعد استخدامها عند الحاجة (مثلًا في ملف water_calculator.dart).
- **الجنس والعمر لا يغيّران النتيجة** في هذا التكليف، لأنه لا توجد معادلة واضحة ومبررة لذلك. يُعرضان فقط كمعلومات أو للتفاعل مع الواجهة.
- اعرض النتيجة برقمين عشريين، مثل 2.95.

هيكل الدالة (من دون الحل)

```
double calculateWaterLitres({
  required int weightKg,
  required double activityHours,
}) {
  // TODO 1: baseMl     = weightKg * 35
  // TODO 2: activityMl = activityHours * 250
  // TODO 3: return (baseMl + activityMl) / 1000
}
```

التحقق من المدخلات

المُدخل	القيمة الافتراضية المقترحة	الحد الأدنى	الحد الأقصى
الوزن (kg)	60	30	200
العمر (years)	20	10	100
ساعات النشاط (h)	1	0	8

- لا تسمح للأزرار (+) و (-) بتجاوز الحدين الأدنى والأقصى.
- عند الضغط على Calculate، تأكد أن الجنس محدد، وأن القيم ضمن الحدود، وإلا اعرض رسالة تنبيه بسيطة (مثل SnackBar أو نص أحمر).

النص التوجيهي في شاشة النتيجة

اعرض نصًا مبسّطًا حسب قيمة النتيجة، بصياغة محايدة بلا ادعاءات طبية (يمكنك تعديل الصياغة بكلماتك):

النتيجة (لتر)	مثال على النص
أقل من 2.0	تقدير منخفض نسبيًا ضمن هذه المعادلة التعليمية. يمكنك توزيع شرب الماء على مدار اليوم.
من 2.0 إلى 3.0	تقدير متوسط ضمن هذه المعادلة التعليمية. حاول توزيع الشرب على فترات خلال اليوم.
أكثر من 3.0	تقدير مرتفع نسبيًا بسبب الوزن أو ساعات النشاط. الاحتياج الفعلي يختلف من شخص لآخر، ويمكنك استشارة مختص عند الحاجة.

5 التصميم ونظام الألوان

لوحة الألوان

العنصر	اللون	الكود
خلفية التطبيق		#0A0E21
البطاقة النشطة		#1D1E33
البطاقة غير النشطة		#111328
لون الماء الأساسي		#00E5FF
النص الرئيسي		#FFFFFF
النص الثانوي		#B0BEC5

متطلبات التصميم

- استخدام Dark Theme مع تباين واضح بين النص والخلفية.
- مسافات متناسقة بين العناصر، وحواف دائرية للبطاقات والأزرار.
- استخدام أيقونات مناسبة.
- مراعاة أحجام الشاشات المختلفة وتجنّب RenderFlex overflow.
- استخدام SafeArea و SingleChildScrollView عند الحاجة.
- استخدام ThemeData لتوحيد الألوان والمظهر.

مقتطف توضيحي: الثيم العام

```
MaterialApp(
  theme: ThemeData.dark().copyWith(
    scaffoldBackgroundColor: const Color(0xFF0A0E21),
    // TODO: primaryColor, appBarTheme, ...
  ),
  home: const InputScreen(),
);
```

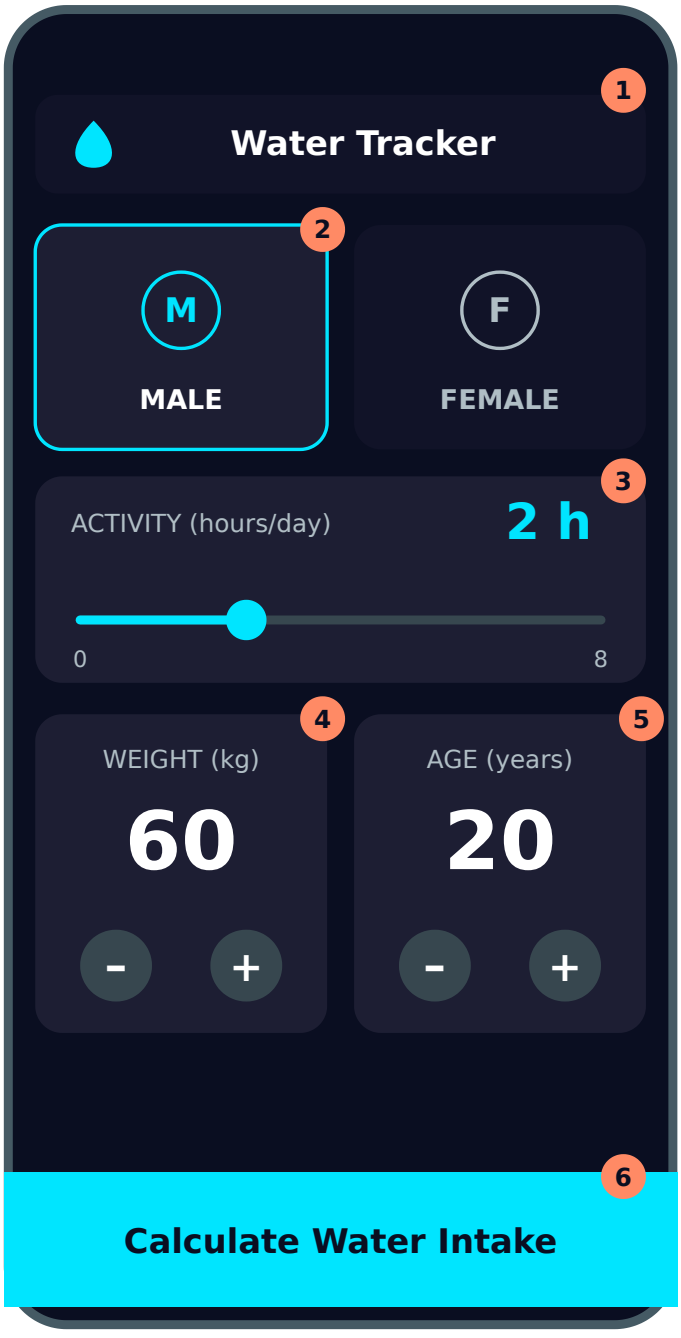
نصيحة

عرّف الألوان في مكان واحد (مثل ثوابت `const Color` في أعلى الملف أو في ملف constants) حتى تغيّرها مرة واحدة فقط عند الحاجة.

6 مخططات Wireframe للشاشتين

المخططات التالية توضّح **توزيع العناصر** فقط؛ يمكنك تغيير الأحجام والتفاصيل بما يناسب تصميمك، مع الالتزام بالعناصر المطلوبة.

الشاشة الأولى: Input Screen



Input Screen — Wireframe

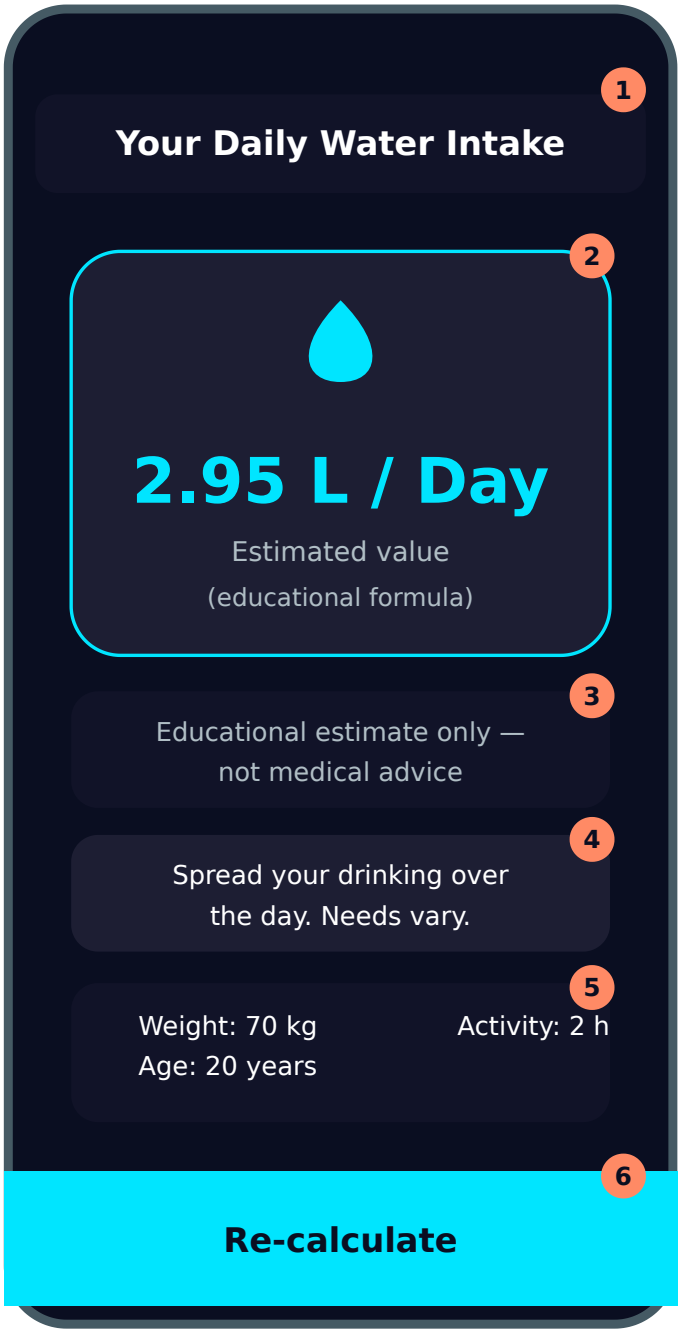
#	العنصر	الوصف
1	AppBar	عنوان التطبيق وأيقونة الماء.
2	Gender Cards	بطاقتا Male و Female؛ البطاقة المحددة بلون وحدّ سماوي.
3	Activity Slider	Slider من 0 إلى 8 مع عرض القيمة الحالية.
4	Weight Card	العنوان + القيمة + زرا (-) و (+) الدائريان.
5	Age Card	العنوان + القيمة + زرا (-) و (+) الدائريان.
6	Bottom Button	زر عريض Calculate Water Intake أسفل الشاشة.

ملاحظة التصميم

بطاقة Male محددة (لون #1D1E33 وحدّ سماوي)، وبطاقة Female غير محددة (لون #111328).

مخطط الشاشة الثانية

الشاشة الثانية: Result Screen



Result Screen — Wireframe

#	العنصر	الوصف
1	AppBar	العنوان Your Daily Water Intake.
2	Result Card	بطاقة مركزية بارزة تعرض القيمة مثل 2.95 L / Day.
3	Disclaimer	توضيح أن القيمة تقديرية وليست توصية طبية.
4	Guidance Text	نص توجيهي مبسّط يعتمد على النتيجة.
5	Input Summary	ملخص الوزن والعمر وساعات النشاط.
6	Bottom Button	زر Re-calculate للعودة إلى الشاشة الأولى.

ملاحظة التصميم

استخدم نفس ألوان الشاشة الأولى. يمكنك إضافة أيقونة ماء داخل بطاقة النتيجة.

كيف تقرأ المخطط؟

الدوائر البرتقالية المرقّمة تشير إلى أقسام الشاشة كما وردت في القسم 3. ابدأ بتقسيم الشاشة إلى Column رئيسي، ثم استخدم Row للعناصر التي تقع جنبًا إلى جنب (مثل بطاقتي الجنس وبطاقتي الوزن والعمر).

7 الـ Widgets المطلوب تطبيقها

يوضح الجدول وظيفة كل Widget ومثالًا على مكان استخدامها. أساسي تستخدمه في كل مشروع تقريبًا، حسب الحاجة تستخدمه عندما تحتاجه لتحسين الشكل أو منع المشاكل.

Widget	وظيفتها في المشروع	مثال لمكان الاستخدام	المستوى
MaterialApp	يهيّئ التطبيق ويحدد الشاشة الرئيسية والثيم.	في main.dart: home و theme.	أساسي
Scaffold	الهيكل الأساسي للشاشة (الخلفية، AppBar، body).	كل شاشة من الشاشتين.	أساسي
AppBar	شريط العنوان في أعلى الشاشة.	عنوان التطبيق وأيقونة الماء.	أساسي
SafeArea	يحمي المحتوى من حواف الشاشة والنتوءات.	داخل body حول المحتوى.	حسب الحاجة
Container	صندوق للتلوين والحواف والأبعاد.	بطاقات الجنس والوزن والعمر.	أساسي
Column	ترتيب العناصر عموديًا.	الشاشة الرئيسية وبطاقة النتيجة.	أساسي
Row	ترتيب العناصر أفقيًا.	بطاقتا Male/Female، وبطاقتا الوزن والعمر.	أساسي
Expanded	يجعل العنصر يملأ المساحة المتاحة بنسبة معينة.	كل بطاقة داخل Row تأخذ نصف العرض.	أساسي
Flexible	يمنح العنصر مرونة دون إلزامه بملء المساحة.	نص طويل داخل Row.	حسب الحاجة
Padding	مسافات داخلية حول العنصر.	حول محتوى البطاقات.	أساسي
SizedBox	مسافة ثابتة بين العناصر أو تحديد حجم.	فاصل بين الأقسام بارتفاع 15.	أساسي
Text	عرض النصوص مع تنسيقها.	عناوين البطاقات والقيم.	أساسي
Icon	عرض أيقونة جاهزة.	أيقونات الجنس والماء.	أساسي
Card	بطاقة جاهزة بحواف وظل (بديل لـ Container).	بطاقة النتيجة في الشاشة الثانية.	حسب الحاجة
Slider	شريط لاختيار قيمة ضمن نطاق.	اختيار ساعات النشاط من 0 إلى 8.	أساسي
SliderTheme	تخصيص ألوان Slider وأحجامه.	جعل المقبض والمسار سماويين.	حسب الحاجة
GestureDetector	التقاط اللمس على أي عنصر.	اختيار بطاقة الجنس عند الضغط.	أساسي
InkWell	تفاعل لمس مع تأثير موجي (بديل لـ GestureDetector).	بطاقات قابلة للضغط.	حسب الحاجة
ElevatedButton	زر مرتفع جاهز.	زر Re-calculate أو زر الحساب.	أساسي
IconButton	زر يحتوي أيقونة (يمكن جعله دائريًا).	أزرار (+) و (-).	أساسي

Center	توسيط العنصر داخل المساحة المتاحة.	بطاقة النتيجة.	أساسي
SingleChildScrollView	يتيح تمرير المحتوى لتجنب Overflow.	حول Column الرئيسي في الشاشة الأولى.	حسب الحاجة
Navigator	التنقل بين الشاشات: push و pop.	الانتقال إلى النتيجة والعودة منها.	أساسي
StatelessWidget	Widget ثابت لا يتغير بعد بنائه.	ResultScreen و GenderCard.	أساسي
StatefulWidget	Widget يتغير بتغير الحالة عبر setState().	InputScreen (الوزن والعمر والجنس).	أساسي
ThemeData	توحيد الألوان والمظهر في التطبيق كله.	Dark Theme في MaterialApp.	حسب الحاجة

ملاحظة

بعض العناصر، مثل RawMaterialButton، اختيارية أو بديلة (مثلًا لصنع زر دائري بدل IconButton). ولا يلزم استخدام جميع الـ Widgets في كل جزء من التطبيق.

8 الهيكلية البرمجية المقترحة

نظّم المشروع بهذه الصورة البسيطة داخل مجلد lib:

```
lib/
├── main.dart
├── screens/
│   ├── input_screen.dart
│   └── result_screen.dart
├── widgets/
│   ├── gender_card.dart
│   ├── counter_card.dart
│   └── bottom_button.dart
└── utils/
    └── water_calculator.dart
```

الملف	وظيفته
`main.dart`	نقطة بداية التطبيق: يشغّل MaterialApp ويحدد الثيم والشاشة الأولى.
`screens/input_screen.dart`	الشاشة الأولى (StatefulWidget): تحتفظ بحالة الجنس والوزن والعمر والنشاط.
`screens/result_screen.dart`	شاشة النتيجة (StatelessWidget): تستقبل البيانات وتعرضها.
`widgets/gender_card.dart`	بطاقة اختيار الجنس القابلة لإعادة الاستخدام (أيقونة + نص + حالة التحديد).
`widgets/counter_card.dart`	بطاقة العداد (الوزن/العمر) بعنوان وقيمة وزري (+) و (-).
`widgets/bottom_button.dart`	الزر العريض أسفل الشاشة (Calculate / Re-calculate).
`utils/water_calculator.dart`	دالة الحساب المستقلة calculateWaterLitres().

مرونة في التنفيذ

يمكنك تنفيذ المشروع في **ملف واحد** إذا كان هذا هو المستوى الذي وصلت إليه. تقسيم المشروع إلى ملفات منفصلة يُعد **ميزة إضافية** تُقدَّر في معيار تنظيم الكود.

مقتطفات توضيحية قصيرة

1) حالة متغيرة داخل StatefulWidget

```dart
class _InputScreenState extends State<InputScreen> {
  int weight = 60;

  void increaseWeight() {
    setState(() {
      // TODO: increase weight but not above the maximum
    });
  }
  // build() ...
}
```

2) الانتقال إلى شاشة النتيجة وتمرير البيانات

```
Navigator.push(
  context,
  MaterialPageRoute(
    builder: (context) => ResultScreen(
      litres: litres,
      weight: weight,
      // TODO: pass age and activityHours too
    ),
  ),
);
// In ResultScreen, Re-calculate button:
// Navigator.pop(context);
```

تذكير

هذه المقتطفات لتوضيح الفكرة فقط وليست الحل الكامل؛ المطلوب أن تكتب بقية الكود بنفسك.

9 التحديات الإضافية Bonus Challenges

مهام اختيارية للطلاب المتميزين:

1. إضافة مؤشر دائري يوضح النتيجة.
2. إضافة رسم أو حركة بسيطة لقطرة الماء.
3. إنشاء Widget مخصص لإعادة استخدام بطاقات البيانات.
4. إضافة زر Reset لإعادة المدخلات إلى قيمها الافتراضية.
5. تحسين التصميم ليتناسب مع أحجام الشاشات المختلفة.
6. حفظ آخر المدخلات محليًا (تحدٍّ إضافي فقط وليس شرطًا أساسيًا).

ملاحظة

هذه التحديات **اختيارية** ولا تؤثر سلبًا على تقييم الطالب الذي يحقق المتطلبات الأساسية.

10 معايير التقييم Rubric

المعيار	ما الذي يُنظر إليه؟	الدرجة
تصميم الشاشة الأولى	وجود كل الأقسام (رأس، جنس، نشاط، وزن، عمر، زر) وتناسق الألوان والمسافات.	20
تصميم شاشة النتيجة	بطاقة النتيجة، التوضيح بأنها تقديرية، النص التوجيهي، ملخص المدخلات.	15
التفاعل مع الأزرار والبطاقات والـ Slider	تحديد الجنس بخيار واحد، تغيّر القيم فعليًا مع setState().	15
صحة الحسابات والتحقق من المدخلات	المعادلة صحيحة وفي دالة مستقلة، ومنع القيم غير المنطقية.	15
التنقل وتمرير البيانات بين الشاشات	استخدام Navigator.push()/pop() وتمرير القيم الصحيحة.	10
تنظيم الكود وإعادة استخدام Widgets	تسمية واضحة، تقسيم إلى Widgets أو ملفات، تجنب التكرار.	10
معالجة أحجام الشاشات ومنع Overflow	عدم ظهور RenderFlex overflow على أحجام مختلفة.	5
README وصور التطبيق وجودة التسليم	README مكتمل، صورتان للشاشتين، المشروع يعمل بلا أخطاء.	10
الإجمالي		**100**

أساس التقييم

يقوم التقييم على **التنفيذ الفعلي وفهم الطالب** للكود الذي كتبه، وليس على جمال التصميم وحده. قد يُطلب منك شرح جزء من الكود (مثل setState() أو طريقة تمرير البيانات).

11 متطلبات التسليم

قدّم ما يلي:

- رابط GitHub Repository.
- الكود المصدري كاملًا.
- ملف README.md يحتوي على وصف المشروع، وطريقة تشغيله، والـ Widgets المستخدمة.
- صورتان على الأقل توضحان الشاشتين.
- توضيح مختصر للمعادلة الحسابية.
- التأكد من أن المشروع يعمل دون أخطاء تمنع التشغيل.

قائمة التحقق قبل التسليم Checklist

- ☐ المشروع يعمل بدون أخطاء تمنع التشغيل (flutter run).
- ☐ الشاشة الأولى تحتوي على: الرأس، بطاقتي الجنس، Slider، بطاقة الوزن، بطاقة العمر، زر الحساب.
- ☐ لا يمكن تحديد بطاقتي الجنس معًا، ويتغير لون البطاقة المحددة.
- ☐ أزرار (+) و (-) تغيّر الوزن والعمر، ولا تتجاوز الحدين الأدنى والأقصى.
- ☐ الحساب يتم داخل دالة مستقلة، والنتيجة صحيحة في حالات الاختبار.
- ☐ شاشة النتيجة تعرض القيمة والتوضيح بأنها تقديرية والنص التوجيهي وملخص المدخلات.
- ☐ زر Re-calculate يعيدني إلى الشاشة الأولى.
- ☐ الألوان متناسقة مع اللوحة المطلوبة (Dark Theme).
- ☐ لا يظهر RenderFlex overflow عند تجربة أحجام شاشات مختلفة.
- ☐ README.md مكتمل ويحتوي على الصورتين وشرح المعادلة والـ Widgets المستخدمة.
- ☐ تم رفع المشروع إلى GitHub والرابط يعمل.

اسم الطالب:	موعد التسليم: .....

12 إرشادات التنفيذ

Suggested Development Steps

1	**إنشاء مشروع Flutter**	flutter create water_tracker ثم شغّل المشروع للتأكد من عمله.
2	**تحديد الألوان والثيم العام**	عرّف الألوان و ThemeData في main.dart.
3	**بناء الشاشة الأولى**	ابدأ بالهيكل (Scaffold, Column, Row) ثم أضف البطاقات.
4	**إضافة التفاعل وتحديث الحالة**	اجعل الشاشة StatefulWidget واستخدم setState() للجنس والوزن والعمر والنشاط.
5	**إنشاء دالة الحساب واختبارها**	اختبرها بحالات الاختبار في القسم 4 قبل ربطها بالواجهة.
6	**بناء شاشة النتيجة**	ابنها كـ StatelessWidget تستقبل القيم عبر المُنشئ (constructor).
7	**تنفيذ التنقل وتمرير البيانات**	Navigator.push() مع تمرير القيم، و Navigator.pop() للعودة.
8	**اختبار الشاشات والأحجام المختلفة**	جرّب أحجام شاشات مختلفة وتأكد من غياب Overflow.
9	**رفع المشروع على GitHub وتجهيز README**	أضف الصور وشرح المعادلة وقائمة الـ Widgets.

الأخطاء الشائعة

الخطأ	العَرَض	الحل
نسيان setState()	تتغير القيمة في الكود لكن الشاشة لا تتغير.	ضع التعديل داخل setState(() { ... }).
تمرير بيانات غير صحيحة	شاشة النتيجة تعرض قيمًا خاطئة أو ترمي خطأ.	تأكد من ترتيب ونوع المعاملات في ResultScreen واطبع القيم للتجربة.
حدوث Overflow	شريط أصفر وأسود يظهر على الشاشة.	استخدم Expanded/Flexible و SingleChildScrollView وقلّل الأحجام الثابتة.
استخدام Navigator بصورة غير صحيحة	الشاشة لا تنتقل أو تظهر شاشة سوداء.	استخدم context الصحيح، و pop() فقط عند وجود شاشة سابقة.
خلط الأنواع (int / double)	خطأ في الحساب أو في عرض الرقم.	حوّل الأنواع صراحة واستخدم toStringAsFixed(2) للعرض.

نصيحة أخيرة

ابنِ المشروع على مراحل صغيرة واختبر كل مرحلة قبل الانتقال للتي بعدها. وإذا واجهتك مشكلة، اقرأ رسالة الخطأ في Debug Console بهدوء؛ غالبًا تحدد لك السطر المسبب للمشكلة.